

Morphic Studio Software Architecture Specification

Document 48 — Plugin, Extension & Marketplace Architecture

Version: 1.0

Status: Approved

1. Purpose

This document defines the architecture for the Morphic Studio Plugin System, Extension Framework, and Marketplace.

Its objective is to allow developers, organizations, and future AI agents to safely extend platform functionality without modifying the core application.

The platform must remain secure, stable, and maintainable while encouraging a healthy extension ecosystem.

2. Platform Philosophy

The extension system follows six core principles.

2.1 Capability-Based Extensions

Plugins register capabilities rather than modifying internal application code.

Examples include:

- AI Providers
- Export Formats
- Importers
- Publishing Targets
- Asset Processors
- Workflow Actions

- Visualization Components
- Analytics Providers
- UI Panels

The core application discovers capabilities dynamically.

2.2 Stable Public Contracts

Plugins communicate only through documented APIs and SDKs.

Internal implementation details remain private.

Breaking changes require versioned compatibility.

2.3 Secure by Default

Plugins execute with explicit permissions.

Access is granted only to approved resources.

Every plugin operates under the principle of least privilege.

2.4 Isolation

Plugins execute independently of the core platform.

Failures within one plugin must not destabilize the application.

2.5 Discoverability

All available capabilities are registered within a central capability registry.

The registry enables dynamic discovery and lifecycle management.

2.6 Future-Proofing

The architecture supports future expansion without requiring redesign.

3. Plugin Categories

Supported categories include:

AI Providers

Model Adapters

Workflow Automation

Publishing Destinations

Importers

Exporters

Asset Processing

Rendering

Visualization

Analytics

Productivity Tools

Collaboration

Enterprise Integrations

Developer Utilities

4. Capability Registry

Each plugin registers:

Plugin ID

Version

Author

Description

Capabilities

Permissions

Dependencies

Compatibility

Lifecycle Hooks

The registry validates every registration.

5. Extension SDK

Provide an official SDK supporting:

TypeScript

Future SDKs:

Python

C#

Java

Swift

Kotlin

SDK responsibilities include:

Authentication

Capability registration

API communication

Configuration

Logging

Version compatibility

Error handling

6. Plugin Lifecycle

Every plugin follows a defined lifecycle.

Install

↓

Verify

↓

Register

↓

Initialize

↓

Execute

↓

Update

↓

Disable

↓

Uninstall

Every transition should be observable and reversible.

7. Permission Model

Plugins request explicit permissions.

Examples include:

Project Access

Asset Access

AI Gateway Access

Export Access

Notifications

Publishing

Analytics

Settings

Users approve requested permissions during installation.

8. Sandboxing

Plugins execute within isolated environments.

Restrictions include:

Filesystem access

Network access

Credential access

Secret access

Memory limits

CPU limits

Execution time limits

Isolation protects platform stability.

9. UI Extension Points

Plugins may extend approved interface regions.

Examples include:

Sidebar panels

Inspector panels

Toolbar actions

Context menus

Dashboard widgets

Settings pages

Command palette

Timeline tools

The core UI controls placement and consistency.

10. AI Extension Model

Plugins may contribute:

AI providers

Prompt templates

Reasoning pipelines

Evaluation tools

Custom capabilities

All AI extensions communicate through the Unified AI Gateway.

11. Workflow Automation

Plugins may define workflow actions.

Examples:

Batch processing

Project automation

Scheduled tasks

Pipeline orchestration

Asset transformations

Approval workflows

Automation integrates with the platform workflow engine.

12. Marketplace

Marketplace supports:

Discovery

Installation

Updates

Ratings

Documentation

Version history

Publisher verification

Categories

Search

Future monetization

Marketplace quality standards should protect users.

13. Compatibility

Plugins declare:

Minimum platform version

Maximum supported version

SDK version

Dependencies

Optional capabilities

Compatibility should be verified before installation.

14. Monitoring

Track:

Plugin usage

Performance

Errors

Resource consumption

Crash reports

Compatibility issues

Permission usage

Users should understand the operational impact of installed plugins.

15. Enterprise Extensions

Future enterprise deployments may install:

Private plugins

Internal integrations

Compliance modules

Security extensions

Custom AI providers

Internal workflow systems

These extensions may remain outside the public marketplace.

16. Acceptance Criteria

The plugin architecture is successful if:

- Core functionality remains independent.
 - Plugins use stable public contracts.
 - Capabilities are discoverable.
 - Permissions are enforced.
 - Failures remain isolated.
 - Marketplace installations are secure.
 - Future platform growth requires minimal architectural change.
-

Conclusion

The Morphe Studio Plugin, Extension, and Marketplace Architecture transforms the platform from a standalone application into an extensible ecosystem. By emphasizing capability-based extensions, stable APIs, sandboxed execution, and a governed marketplace, the architecture enables sustainable innovation while preserving the security, reliability, and maintainability of the core platform.